

CÍVICA SOFTWARE

Sistema de gestión de incidencias · Turno de recuperación

Cuadernillo del desarrollador de turno

Prueba práctica de recuperación · Corte 1 · Algoritmos y estructura de datos

Nombre completo	Julián David Hernández León	Fecha	2/09/26
-----------------	-----------------------------	-------	---------

Este cuadernillo contiene todo lo que usted necesita durante el turno: la mecánica de la prueba, las instrucciones detalladas de los tres tickets y su hoja de ruta. Consérvelo: es el documento que se entrega en la mesa de despacho al cerrar el turno.

Cómo funciona este turno

Este es un turno de recuperación. Son tres tickets en lugar de cinco, pero cubren los mismos temas del Corte 1: arreglos y matrices, memoria dinámica y punteros, programación orientada a objetos, y listas enlazadas.

La mecánica es la misma que ya conoce: un ticket queda cerrado cuando su código pasa las pruebas automáticas, y al pasar aparece un código de cierre que usted anota en la hoja de ruta.

Momento	Minuto	Actividad	Duración
1	0 – 10	Briefing del turno y revisión de la carpeta de trabajo	10 min
2	10 – 35	TCK-5510 · P3 — Mapa de cobertura con datos incompletos	25 min
3	35 – 65	TCK-5511 · P2 — Registro de sanciones: memoria y tipos	30 min
4	65 – 90	TCK-5512 · P0 — Historial corrupto: producción caída	25 min

Duración total del turno: 90 minutos. Las ventanas son una referencia de ritmo, no un plazo: si cierra un ticket después de que su ventana pasó, cuenta igual.

El ciclo de trabajo

1. **Lea el ticket completo:** el reporte de quien lo abrió y el criterio de aceptación.
2. **Corrija únicamente el archivo indicado** en la ficha del ticket.
3. **Ejecute las pruebas automáticas.** Le muestran qué pasa y qué falla. Puede ejecutarlas cuantas veces quiera.
4. **Cuando todas pasen, aparece el CÓDIGO DE CIERRE.** Anótelos en su hoja de ruta.

SOBRE EL CÓDIGO DE CIERRE No está escrito en ningún archivo: se calcula a partir de los resultados correctos. No se puede adivinar, ni leer del archivo de pruebas, ni copiar del compañero.

Sus archivos y sus comandos

```
prueba_recuperacion/estudiante/
TCK-5510/   cobertura.py   pruebas_t1.py
TCK-5511/   sanciones.cpp  pruebas_t3.py
TCK-5512/   historial.py
```

Ticket	Usted modifica	Usted ejecuta
TCK-5510	cobertura.py	python3 pruebas_t1.py
TCK-5511	sanciones.cpp	g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp ./sanciones
TCK-5512	historial.py	python3 pruebas_t3.py

REGLA QUE NO TIENE EXCEPCIÓN Los archivos cuyo nombre empieza por «pruebas» NO se modifican. Alterarlos para que impriman el código es fraude y aplica el protocolo del reglamento.

Su nota

La nota es la suma de los tickets que cierre. Los tickets son independientes: puede resolverlos en cualquier orden y saltarse el que se le atragante. Puede cerrar su turno y salir cuando quiera.

Ticket	Sev	Vale	Si cierra hasta aquí y sale	Resultado
TCK-5510	P3	1.5	1.5	Insuficiente
TCK-5511	P2	1.7	3.2	Aprueba
TCK-5512	P0	1.8	5.0	Turno completo

- **El TCK-5511 es el que decide la aprobación.** Con los dos primeros cerrados usted ya aprueba.
- **El TCK-5512 vale más que los otros.** Si le queda poco tiempo y no ha cerrado nada, es el que más rinde.

La mesa de despacho

1. Cuando decida cerrar su turno, lleve este cuadernillo a la mesa.
2. La profesora le pedirá que ejecute las pruebas en su pantalla, delante de ella. No sirve una captura ni una foto.
3. Coteja los códigos, firma su hoja de ruta y usted se retira.

Reglas del turno

- Trabajo individual. Cada uno en su equipo.
- Material permitido: únicamente la documentación oficial de C++ y de Python.
- Puede ejecutar las pruebas cuantas veces quiera. No hay penalización por intentar.
- Si su código no compila, lea el error completo antes de levantar la mano.
- Una vez firma en la mesa, el turno terminó: no se puede volver a entrar.

TICKETS RESUELTOS DE prueba_corte1:

TCK-4420

TCK-4421

TCK-4422

SE CAMBIA EL TCK-4423 DE prueba_corte1 POR EL TCK-5511 DE PARTE2

PARTE2 TICKETS RESUELTOS:

TCK-5511

TCK-5512

LINK DE GITHUB:

https://github.com/worksedu-prog/Trabajos-AlgoritmosS2/tree/385f461abd495e813c04a926b140c4aff68db587/prueba_corte1

TCK-5511 Registro de sanciones: no distingue tipos y consume memoria**SEVERIDAD P2** · PrestaLab · ventana 35–65 min · C++ · 1.7 puntos

Reportado por: Coordinación de préstamos e Infraestructura

«El listado imprime «Usuario» para todos. Necesitamos distinguir estudiantes de usuarios externos.»

«Las sanciones del primer incidente de cada usuario no se están contando: el total sale corto.»

«Y el proceso arranca en 40 MB y termina el día en 600 MB. Hay que reiniciarlo cada noche.»

Contexto del sistema

El archivo sanciones.cpp gestiona el registro de sanciones. Cada objeto Usuario reserva memoria dinámica para su arreglo de días de sanción. Hay una clase base y faltan dos clases especializadas.

SON TRES PROBLEMAS DISTINTOS Este ticket combina lógica, polimorfismo y memoria. Resuelva primero la lógica y las clases: son las que destraban el código de cierre. La memoria se verifica aparte, con el sanitizer.

Qué debe hacer

1. **Corregir el defecto de lógica en totalDias().** El recorrido empieza en la posición 1 y por eso se pierde el primer incidente de cada usuario.
2. **Crear la clase Estudiante,** que herede de Usuario y agregue el programa académico. Su descripción debe devolver exactamente: Estudiante ES-002 de Ingeniería
3. **Crear la clase Externo,** que herede de Usuario y agregue la entidad de procedencia. Su descripción debe devolver exactamente: Externo EX-003 (Alcaldía)
4. **Activar las líneas comentadas de main,** tanto las que crean los objetos como las que registran sus sanciones.
5. **Hacer que cada objeto responda con SU propia descripción** aunque se recorra el arreglo con punteros a la clase base. Si todos imprimen «Usuario», falta una palabra clave.
6. **Cerrar las fugas de memoria.** Busque los comentarios FALTA ALGO AQUI y FALTAN LIBERACIONES AQUI. Recuerde que una clase base con clases hijas necesita destructor virtual.

Criterio de aceptación

- El programa imprime las tres descripciones, cada una con su formato propio.
- El programa imprime TOTAL=12 y a continuación el código de cierre.
- Compilado con -fsanitize=address, la ejecución NO produce ningún bloque ERROR: AddressSanitizer.
- Se verifican las dos últimas cosas en la mesa de despacho. Una sola no basta.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5511
$ g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp
$ ./sanciones
```

El texto de las descripciones se compara carácter por carácter. Respete mayúsculas, espacios y paréntesis exactamente como se indican arriba.

Errores frecuentes en este ticket

- Olvidar virtual en descripcion(): todos los objetos imprimen la descripción de la clase base.
- Olvidar el destructor virtual: el sanitizer reporta fuga al liberar por puntero a la base.
- Liberar el arreglo de punteros antes que los objetos que contiene: se pierden las direcciones.
- Activar la creación de los objetos pero olvidar activar las líneas que registran sus sanciones: el total queda corto.

Código de cierre obtenido: _____

MODIFICACIONES:

Línea 34: Se cambia la condición de for, en vez de `int = 1` cambia a `int = 0`. Porque el primer incidente también cuenta en esta condición

Línea 27: Se agrega virtual `~Usuario() { delete[] dias; }` Para evitar una fuga de memoria

Línea 42 – 47: Se crea la clase Estudiante (hereda de Usuario, agrega el programa academico) con `descripcion()` debe devolver: "Estudiante " + código + " de " + programa

Línea 50 – 55: Se crea la clase Externo (hereda de Usuario, agrega la entidad de procedencia) con `descripcion()` debe devolver: "Externo " + código + " (" + entidad + ")"

Línea 83 – 86: Se agrega un destructor para evitar fuga de memoria

CODIGO:

```
// =====
// Cívica Software · TCK-5511 · Severidad P2
// Sistema: PrestaLab — Registro de sanciones de usuarios
// Compile SIEMPRE con:
// g++ -std=c++17 -fsanitize=address -g -o sanciones sanciones.cpp
//
// Dos problemas reportados:
// 1. El listado imprime "Usuario" para todos, sin distinguir el tipo.
// 2. Las sanciones del PRIMER incidente no se estan contando.
// 3. El proceso consume memoria sin parar.
// =====

#include <iostream>

#include <string>

using namespace std;

class Usuario {
protected:
    string codigo;
    int* dias; // dias de sancion acumulados por incidente
    int n;
public:
    Usuario(string c, int cantidad) : codigo(c), n(cantidad) {
```

```
dias = new int[n];

for (int i = 0; i < n; i++) dias[i] = 0;
}

virtual ~Usuario() { delete[] dias; }

//(SE AGREGA VIRTUAL AL DESTRUCTOR)

void sancionar(int i, int d) { if (i >= 0 && i < n) dias[i] = d; }

int totalDias() const {
    int s = 0;
    for (int i = 0; i < n; i++) s += dias[i];  // (SE CAMBIA EL INICIO A 0)
    return s;
}

string getCodigo() const { return codigo; }

string descripcion() const { return "Usuario " + codigo; }
};

class Estudiante : public Usuario {
    string programa;
public:
    Estudiante(string c, int cantidad, string p) : Usuario(c, cantidad), programa(p) {}
    string descripcion() const { return "Estudiante " + codigo + " de " + programa; }
};

//Se crea la clase Estudiante (hereda de Usuario, agrega el programa academico)
//    descripcion() debe devolver: "Estudiante " + codigo + " de " + programa

class Externo : public Usuario {
    string entidad;
public:
    Externo(string c, int cantidad, string e) : Usuario(c, cantidad), entidad(e) {}
```

```
string descripcion() const { return "Externo " + codigo + " (" + entidad + ")"; }

};

// Se crea la clase Externo (hereda de Usuario, agrega la entidad de procedencia)

//      descripcion() debe devolver: "Externo " + codigo + " (" + entidad + ")"

int main() {

    const int N = 3;

    Usuario** registro = new Usuario*[N];

    for (int i = 0; i < N; i++) registro[i] = nullptr; // sin basura en el arreglo

    registro[0] = new Usuario("US-001", 3);

    registro[1] = new Estudiante("ES-002", 3, "Ingenieria");

    registro[2] = new Externo("EX-003", 3, "Alcaldia");


    registro[0]->sancionar(0, 2);

    registro[1]->sancionar(0, 5); registro[1]->sancionar(1, 1);

    registro[2]->sancionar(2, 4);


    int suma = 0;

    for (int i = 0; i < N; i++) {

        if (registro[i] == nullptr) continue;

        cout << registro[i]->descripcion() << " -> " << registro[i]->totalDias() << " dias" << endl;

        suma += registro[i]->totalDias();

    }

    cout << "TOTAL=" << suma << endl;


    if (suma == 12) // el codigo se DERIVA del total correcto

        cout << "TICKET CERRADO - codigo de cierre: 5511-" << suma << N << endl;


    // (SE AGREGA EL DESTRUCTOR PARA EVITAR FUGA DE MEMORIA)

    for (int i = 0; i < N; i++) {

        delete registro[i];
```

```
}  
  
delete[] registro;  
  
return 0;  
}
```

IMPRIME:

```
Usuario US-001 -> 2 días  
Usuario ES-002 -> 6 días  
Usuario EX-003 -> 4 días  
TOTAL=12  
TICKET CERRADO - codigo de cierre: 5511-123  
PS C:\Users\Windows\Desktop\Programacion\Repo\Trabajos-AlgoritmosS2>
```


TCK-5512 PRODUCCIÓN CAÍDA — el historial está corrupto**SEVERIDAD PO** · Turno justo · ventana 65–90 min · Python · 1.8 puntos

Reportado por: Mesa de ayuda · tres reportes en la última hora

«Registré la primera atención del día y el sistema se cayó.»

«Deshice la última atención y se borró todo el historial.»

«Busco un turno que sí existe y me dice que no está.»

Contexto del sistema

El archivo historial.py implementa el historial de atenciones de un punto de servicio mediante una lista enlazada simple. Cada nodo guarda un número de turno y el módulo que lo atendió. Los métodos cuantas y listar funcionan bien.

Qué debe hacer

1. **Corregir registrar().** Se cae cuando el historial está vacío, porque intenta recorrer desde una cabeza que no existe. Falta el caso de la lista vacía: cuando no hay cabeza, el nuevo nodo ES la cabeza.
2. **Corregir deshacer_ultima().** Debe eliminar únicamente el ÚLTIMO nodo, no todo el historial. Recuerde que para desenganchar el último hay que llegar al PENÚLTIMO. Y maneje aparte el caso de un solo elemento, donde sí se vacía el historial.
3. **Implementar buscar(turno).** Debe recorrer el historial y devolver el MÓDULO que atendió ese turno, o None si el turno no existe.

NO CAMBIE LAS FIRMAS Los métodos deben seguir llamándose registrar, deshacer_ultima y buscar, y recibir los mismos parámetros. El archivo de pruebas los llama por su nombre.

Criterio de aceptación

- Las ocho verificaciones de pruebas_t3.py en OK.
- Registrar en un historial vacío funciona y no lanza excepción.
- Deshacer la última atención deja el resto del historial intacto.
- Deshacer con un solo elemento deja el historial en cero.
- Deshacer en un historial vacío devuelve False sin lanzar excepción.
- Buscar un turno existente devuelve su módulo; buscar uno inexistente devuelve None.

Cómo se verifica

```
$ cd prueba_recuperacion/estudiante/TCK-5512
$ python3 pruebas_t3.py
```

EL VALIDADOR NO SE CAE Si su código lanza una excepción, el archivo de pruebas la captura y la reporta como FALLA, y sigue con las demás verificaciones. Así usted ve el panorama completo en cada ejecución.

Sugerencia de método

Antes de tocar el código, dibuje en el espacio de abajo cuatro nodos enlazados y marque con una flecha cuál referencia debe cambiar al eliminar el último. Después dibuje el caso de un solo nodo. Este ticket se resuelve más rápido dibujando que probando.

Código de cierre obtenido: _____

MODIFICACIONES:

Línea 27 - 29: Se crea una condición de if actual is None: self.cabeza = Nuevo return. Se crea esta condición pues SI no hay cabeza, el while falla y el actual se convierte en un None

Línea 52 – 57: Se crea un modulo para recorrer la lista. Con la condición de while actual is not None:

Línea 40 – 47:

CODIGO:

```
# =====
# Cívica Software · TCK-5512 · Severidad P0 · PRODUCCION CAIDA
# Sistema: TurnoJusto — El historial de atenciones esta corrupto.
#
# Reportes de soporte:
# - "Registre la primera atencion del dia y el sistema se cayo."
# - "Deshice la ultima atencion y se borro todo el historial."
# - "Busco un turno que si existe y me dice que no esta."
# =====
```

class Nodo:

```
def __init__(self, turno, modulo):
    self.turno = turno
    self.modulo = modulo
    self.siguiente = None
```

class Historial:

```
def __init__(self):
    self.cabeza = None
```

```
def registrar(self, turno, modulo):
```

```
    """Agrega una atencion al FINAL del historial.
```

```
    BUG: se cae cuando el historial esta vacio."""
```

```
    nuevo = Nodo(turno, modulo)
```

```
    actual = self.cabeza          # <-- ¿que pasa si no hay cabeza? (SI NO HAY CABEZA, actual es None y el while falla)
```

```
    if actual is None:
```

```
        self.cabeza = nuevo
```

```
        return
```

```
    while actual.siguiente is not None:
```

```
        actual = actual.siguiente
```

```
    actual.siguiente = nuevo
```

```
def deshacer_ultima(self):
```

```
    """Elimina la ULTIMA atencion registrada.
```

```
    Devuelve True si elimino algo, False si el historial estaba vacio.
```

```
    BUG: borra todo el historial."""
```

```
    if self.cabeza is None:
```

```

    return False
if self.cabeza.siguiente is None:
    self.cabeza = None
    return True
actual = self.cabeza
while actual.siguiente.siguiente is not None:
    actual = actual.siguiente
actual.siguiente = None #(SE AGREGA UNA LINEA PARA ELIMINAR EL ULTIMO NODO)
return True

```

```

def buscar(self, turno):
    """Devuelve el modulo que atendio ese turno, o None si no existe.
    PENDIENTE: implementar."""
    actual = self.cabeza
    while actual is not None:
        if actual.turno == turno:
            return actual.modulo
        actual = actual.siguiente #(SE CREA UN MODULO PARA RECORRER LA LISTA)
    return None

```

```

def cuantas(self):
    n = 0
    actual = self.cabeza
    while actual is not None:
        n += 1
        actual = actual.siguiente
    return n

```

```

def listar(self):
    r = []
    actual = self.cabeza
    while actual is not None:
        r.append(actual.turno)
        actual = actual.siguiente
    return r

```

IMPRIMIDO:

```

PS C:\Users\Windows\Desktop\Programacion\Repo\Trabajos-AlgoritmosS2> & C:\Users\Windows\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/Windows/Desktop/Programacion/Repo/Trabajos-AlgoritmosS2/prueba2_corte1/pruebas_t3.py
TICK-5512 · verificacion
[OK ] registrar en historial VACIO: obtuvo [101]
[OK ] registrar cuatro atenciones: obtuvo [101, 102, 103, 104]
[OK ] deshacer la ULTIMA: obtuvo [101, 102, 103]
[OK ] deshacer con UN solo elemento: obtuvo 0
[OK ] deshacer en historial VACIO: obtuvo False
[OK ] buscar turno existente: obtuvo A
[OK ] buscar turno inexistente: obtuvo None
[OK ] deshacer dos veces seguidas: obtuvo [101, 102]

TICKET CERRADO – codigo de cierre: 5512-34C

```

Hoja de ruta del turno

Anote aquí el código de cierre de cada ticket que logre cerrar. Este es el documento que se entrega en la mesa de despacho.

Ticket	Sev	Sistema	Código de cierre obtenido	Puntos	Visto bueno
TCK-5510	P3	RedAcopio		1.5	
TCK-5511	P2	PrestaLab		1.7	
TCK-5512	P0	TurnoJusto		1.8	

Hora de cierre de turno: _____ Total de puntos: _____

Firma de despacho: _____

Antes de entregar, verifique

- Escribió su nombre en la portada.
- Copió cada código de cierre completo, tal como aparece en pantalla.
- Dejó los archivos guardados en su equipo, sin borrar nada.
- Tiene las pruebas listas para ejecutarlas delante de la profesora.